Autonomous Articulated Vehicle Control via Deep Reinforcement Learning and High-Dimensional Perception

Benjamin Schnapp@inbookID, author = author, title = title, booktitle = booktitle,

March 31, 2026

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The increasing demand for autonomous logistics and the growth of large-scale goods transportation have positioned articulated tractor-trailer vehicles as a critical target for autonomous control research. The transition from rigid-body autonomous navigation to the control of articulated tractor-trailer systems represents a significant escalation in mechanical complexity, perception requirements, and control instability, particularly during reverse maneuvering.

Unlike single-body vehicles, tractor-trailer systems introduce non-holonomic constraints at the hitch point that fundamentally alter the stability characteristics of the vehicle. In reverse motion, the articulation angle γ between the tractor and trailer exhibits open-loop instability: absent precise closed-loop control, any perturbation grows unbounded, culminating in the well-known jackknife condition from which no steering correction can recover the trailer's alignment. Classical approaches such as Nonlinear Model Predictive Control (NMPC) can manage this instability by optimising over a prediction horizon, but their computational demands and reliance on accurate system models limit their applicability in unstructured or sensor-rich environments.

1.2 Problem Definition

The core challenge addressed in this thesis is twofold. First, the kinematic and dynamic complexity of tractor-trailer systems makes it difficult to design reward functions and control policies that simultaneously optimise path tracking performance and hitch stability. Second, real-world autonomous deployment requires perception of lane boundaries and obstacles directly from high-dimensional sensor data, rather than from pre-computed symbolic state representations.

Existing learning-based path-following studies on rigid-body vehicles have shown that

continuous-control reinforcement learning can achieve performance competitive with classical baselines under nominal conditions, but robustness degrades when the vehicle model, operating conditions, or path geometry become more demanding. Those limitations are amplified for articulated systems, where reverse instability and hitch-angle growth impose a much narrower margin for control error.

1.3 Thesis Contributions

This thesis addresses these challenges through the following contributions:

- developing a kinematic and dynamic model of a tractor-trailer system, including hitch instability analysis for reverse manoeuvring;
- designing a vision-based RL observation space using CNN encoders and autoencoders operating on bird's-eye-view (BEV) images rendered from a custom Pygame simulation environment;
- engineering a reward function that balances cross-track error, heading error, hitch articulation stability, and collision avoidance;
- implementing and benchmarking Twin Delayed Deep Deterministic Policy Gradient (TD3) agents in lane-following and obstacle-avoidance tasks within a custom Gymnasium-based simulation environment;
- comparing the learned policies against tuned classical control baselines, including pure-pursuit, PID, and MPC, on matched articulated path-tracking tasks.

The research aligns with the objectives of the Mechatronics Vehicle Systems (MVS) Lab and the WATonoBus project at the University of Waterloo.

1.4 Institutional Context

This work is submitted in partial fulfilment of the requirements for a Master of Applied Science (MASc) degree in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo. The degree requires six terms of full-time registration, completion of 3–5 graduate-level courses, appointment of two additional faculty readers, and a mandatory 15-day public display period. The target completion date is early July, with a seminar scheduled for late July.

Table 1.1: MASc thesis requirements summary

Element	Requirement
Research Scope	Substantial problem extending beyond existing laboratory results
Literature Review	Comprehensive assessment of classical control and modern AI approaches
Validation	Simulation-based empirical evidence across multiple environment configurations
Length/Format	Typically 80–120 pages; GSPA formatting and UWSpace submission
Committee	Supervisor plus two faculty readers; formal display period

1.5 Thesis Organisation

The remainder of this thesis is structured as follows. Chapter 2 reviews classical and learning-based control methods for articulated vehicles, alongside relevant perception and safety literature. Chapter 3 derives the state-space equations for the tractor-trailer system and analyses the jackknife instability. Chapter 4 presents the CNN and autoencoder-based perception stack. Chapter 5 defines the MDP formulation and details the TD3 learning framework used in this work. Chapter 6 describes the custom Pygame/Gymnasium simulation environment and experimental setup. Chapter 7 presents and discusses experimental results. Chapter 8 summarises the findings and outlines future work.

Chapter 2

Literature Review

2.1 Articulated Vehicle Control

2.1.1 Classical Stabilisation Techniques

Classical articulated vehicle control methods address the hitch instability through structured model-based approaches:

- **PID control**: Simple to implement and computationally negligible, but incapable of anticipating the jackknife condition. Gains must be re-tuned for different payloads and path curvatures.
- **Model Predictive Control (MPC)**: Optimises steering over a receding horizon subject to state and input constraints. Linear MPC can enforce hard bounds on the articulation angle γ.
- **Nonlinear MPC (NMPC)**: Captures the full nonlinear kinematics including reverse instability. Optimises steering over a receding horizon subject to state and input constraints, enabling hard bounds on the articulation angle γ, but with higher computational cost than linear methods.
- **Active hitch stabilisation**: Applies an additional actuated degree of freedom at the hitch to damp articulation oscillations. Research at Waterloo into trailer sway control and active hitches demonstrates that lateral articulation at the hitch can mitigate oscillations, but for a standard autonomous tractor-trailer this must be managed purely through the tractor's steering angle δ.

2.1.2 Kinematic Models

The bicycle model provides a baseline for rigid-body longitudinal and lateral dynamics. For an articulated system, the state vector must be expanded to include the trailer heading and articulation angle, as detailed in Chapter 3.

2.2 Reinforcement Learning in Autonomous Driving

2.2.1 Policy Gradient Methods

Deep Deterministic Policy Gradient (DDPG) is an off-policy actor-critic algorithm designed for continuous action spaces. It is one of the canonical deterministic policy-gradient methods used for end-to-end vehicle control, but it is known to suffer from critic overestimation bias and training instability.

Proximal Policy Optimisation (PPO) offers improved stability over DDPG through clipped probability ratios, but its on-policy sample efficiency is lower, which is a significant consideration when simulation rollouts are expensive.

2.2.2 Transformer-Based Methods

Decision Transformers (DT) reformulate reinforcement learning as sequence modelling: an action is predicted from a context window of past states, actions, and returns-to-go using a causal self-attention mechanism. In vehicle-control settings, DT-style models can represent longer temporal dependencies than shallow actor-critic policies, but their performance depends strongly on the diversity and quality of the offline trajectory dataset. In particular, datasets dominated by nominal path-tracking behaviour can lead to poor recovery performance from high-error states. Effective DT training for articulated vehicles would therefore require explicit coverage of recovery manoeuvres and unstable reverse-driving configurations.

2.3 Perception and Representation Learning

2.3.1 Convolutional Neural Networks for Driving

CNNs have become the standard front-end for autonomous driving perception, extracting spatial features from camera images and occupancy grids. End-to-end learning approaches train CNNs directly from sensor inputs to control outputs, allowing the network to learn which visual features are most relevant for the control task. In the simulation setting of this thesis, CNNs process bird's-eye view images rendered from the Pygame environment to extract lane geometry and obstacle information.

2.3.2 Autoencoders for State Compression

Autoencoders compress high-dimensional observations into a compact latent representation, mitigating the curse of dimensionality for RL training. By pre-training an autoencoder on images collected from simulation rollouts, the encoder learns to capture

essential geometric features of the road and obstacles. The RL agent then operates on the latent vector z, which reduces policy network parameters, accelerates convergence, and improves generalisation across environments.

2.4 Safety in Autonomous Systems

Safety of the Intended Functionality (SOTIF) defines the safety boundary for autonomous systems operating under nominal sensor and actuator conditions. For articulated vehicles, a key safety concern is the jackknife condition: the RL reward function must penalise large articulation angles to discourage the agent from entering unstable configurations. Control Barrier Functions (CBFs) offer a formal mechanism to overlay hard safety constraints on top of a learned policy, and are identified as a direction for future work beyond the simulation-based experiments of this thesis.

2.5 Benchmarking Against Recent Waterloo Theses

To contextualise the expected level of rigor, Table 2.1 summarises recent MASc theses from the Mechatronics domain at the University of Waterloo.

Table 2.1: Selected recent MASc theses, University of Waterloo Mechatronics Engineering

Author (Year)	Topic	Methodology	Rigor Highlights
Joseph (2025)	Gaze-enabled grasping assistance	ROS2, Kinova Arm, Meta Quest Pro	User study with 30 participants; workload and performance metrics
Thibault (2022)	Humanoid robotic manipulation	EUROBENCH, REEM-C robot	Defined "loco-manipulation"; new manipulability–stability metrics
Hart (2023)	Gait detection via machine learning	Wearable sensors, ternary classification	6 classical + 1 Transformer model; grid-search hyperparameter optimisation
Wei (2021)	Path following for manipulators	Transpose Jacobian control	Free of inverse transformations; validated on 2-DoF and 4-DoF robots
Bickford (2019)	Autonomous bicycle stabilisation	Linear control, Whipple model	Multi-loop architecture; "virtual paths" for high-error recovery

Common themes include strong mathematical modelling foundations and comprehensive validation datasets. For this thesis, a simple "it works" result is insufficient; the work must quantify stability limits, evaluate performance across varied paths, and compare against classical benchmarks.

Chapter 3

Mathematical Modeling and Problem Formulation

3.1 Tractor Dynamics

The tractor longitudinal and lateral dynamics are:

$$\dot{x} = v_x \cos\psi - v_y \sin\psi \tag{3.1}$$

$$\dot{y} = v_x \sin\psi + v_y \cos\psi \tag{3.2}$$

where v_x is the longitudinal velocity, v_y is the lateral velocity, and ψ is the vehicle heading.

The tractor is modelled as a planar bicycle model, which captures the dominant lateral and yaw behaviour required for lane-following and low-speed manoeuvring. In this representation the front axle is replaced by a single steerable wheel and the rear axle by a single non-steered wheel. The global position (x, y) evolves from the body-frame longitudinal and lateral velocities (v_x, v_y) and the tractor heading ψ, as shown in Equations 3.1 and 3.2. This state description is sufficient for the control tasks studied in the thesis because the dominant quantities of interest are path-relative position error, heading error, steering angle, and hitch articulation.

The modelling objective is not to reproduce every high-speed transient of a road vehicle, but to provide a physically meaningful control environment in which steering commands, heading evolution, and articulation dynamics remain coupled. This balance is important for reinforcement learning: an overly simplified kinematic model would make the task unrealistically easy, whereas a full high-fidelity multibody model would obscure the core control-design questions addressed here.

3.2 Trailer Articulation and Reverse Instability

The articulated system introduces hitch geometry defined by:

- L_h: distance from tractor rear axle to hitch point
- L_t: distance from hitch point to trailer axle
- γ: articulation angle between tractor and trailer

The hitch angle kinematics during reverse motion are:

$$\dot{\gamma} = \frac{v}{L_t} \sin \gamma - \frac{v}{L_h} \tan \delta \cos \gamma \tag{3.3}$$

Reverse motion introduces instability as γ approaches the jackknife threshold.

The trailer introduces an additional geometric state, the articulation angle γ, which measures the relative orientation between the tractor and trailer bodies. This variable is central to articulated-vehicle control because it determines whether the trailer remains aligned with the tractor or diverges toward a jackknife configuration. Equation 3.3 captures how steering angle and longitudinal motion interact through the hitch geometry to change γ over time.

In forward motion, the hitch dynamics are naturally self-correcting over a broad operating range: modest articulation tends to decay as the tractor pulls the trailer back into alignment. In reverse motion, the sign of the longitudinal velocity changes this behaviour qualitatively. Small articulation errors can grow rather than decay, so the driver or controller must actively stabilise the hitch while simultaneously tracking the path. This instability is the defining challenge of reverse articulated manoeuvring and motivates the use of explicit hitch-angle penalties, trailer-referenced tracking metrics, and dedicated reverse-controller formulations throughout the thesis.

3.3 Challenges of Reverse Maneuvering

Reverse motion is characterised by a sign reversal in the kinematic equations, which transforms the stabilising effect of forward velocity into a destabilising one. Successful reverse manoeuvring therefore requires anticipatory control: the steering input must correct developing trailer error before the hitch angle grows large enough to become unrecoverable. In practical terms, the controller must regulate three coupled objectives at once: trailer path tracking, tractor placement, and articulation stability.

This coupling is what distinguishes articulated reverse control from conventional rigid-vehicle lane following. A controller that focuses only on tractor cross-track error can produce apparently reasonable short-term motion while simultaneously driving the hitch

toward instability. For that reason, the reinforcement-learning rewards and classical benchmark metrics used in this thesis prioritise trailer tracking and hitch-angle containment rather than treating them as secondary diagnostics.

3.4 MDP Formulation

The control problem is cast as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma_d)$ where:

- $\mathcal{S}$: observation space consisting of a vector of state variables (steering angle, articulation angle, cross-track error, heading error) and an 84×84 grayscale BEV image;
- $\mathcal{A}$: continuous action space $[\dot{\delta}, v] \in [-15°/\text{s}, 15°/\text{s}] \times [0, 2\,\text{m/s}]$;
- $\mathcal{T}$: transition dynamics simulated in the custom Pygame/Gymnasium environment;
- $\mathcal{R}$: reward function defined in Chapter 5;
- γ_d: discount factor.

Episodes terminate on collision, jackknife ($|\gamma| > 90°$), or successful path completion.

Chapter 4

Perception and Observation Space Design

4.1 Observation Modalities

The simulator now supports three observation families so that perception can be studied as an ablation rather than a fixed design choice:

- **State-only**: an 8-dimensional vector containing steering angle, hitch angle, tractor cross-track and heading error, trailer cross-track and heading error, and two local curvature terms.
- **State + lidar**: the same 8-dimensional state vector augmented with N normalised range readings, where $N \in \{8, 16, 24\}$ in the current Phase 1 experiments.
- **State + BEV image**: a dictionary observation with the state vector and an 84×84 grayscale bird's-eye-view image.

This design allows the thesis to compare hand-engineered geometric sensing against learned visual representations within the same tractor-trailer task.

4.2 Bird's-Eye View Observation

The BEV image is generated by a configurable camera anchored to the tractor's rear axle, rendering a top-down 84×84 grayscale image of the vehicle and its surroundings from the Pygame simulation. The anchor point and zoom level are configurable in the simulation settings. The image encodes lane markings, obstacle positions, local path geometry, and obstacle placement, providing spatial context that the vector observation alone cannot capture.

Forward and reverse variants of the BEV environment share the same image format. In reverse manoeuvres, the policy still receives the same image representation, but the vector

component is interpreted with trailer-led tracking errors because the trailer becomes the leading unit during backward motion.

4.3 CNN Encoder

A Convolutional Neural Network (CNN) serves as the front-end of the RL agent, processing the BEV image to extract spatial features relevant to lane tracking and obstacle avoidance. The CNN enables the agent to perceive the environment directly from rendered images, without requiring a pre-defined symbolic representation of every object.

The CNN architecture is implemented as a custom feature extractor within the Stable Baselines3 framework. A NatureCNN-style image encoder maps the grayscale BEV image to a 3136-dimensional feature vector, while a separate two-layer multilayer perceptron embeds the vector state to 64 dimensions. The two branches are concatenated before being passed to the TD3 actor and critic networks. This combined observation allows the policy to leverage both precise geometric state information and rich spatial context.

The CNN is trained end-to-end with the TD3 agent, allowing the feature extractor to learn which visual features are most relevant for path maintenance and collision avoidance.

4.4 Autoencoder Latent Space

To improve RL training efficiency, an autoencoder can be used to compress high-dimensional BEV image observations into a compact latent representation. The encoder $E(\cdot)$ maps the image observation s to a latent vector z, and the decoder $D(\cdot)$ reconstructs the original image:

$$z = E(s) \tag{4.1}$$

$$\hat{s} = D(z) \tag{4.2}$$

The autoencoder is pre-trained on images collected from simulation rollouts, allowing the encoder to learn compact representations of the road geometry and obstacle configurations. BEV images are collected from a mixture of simple pure-pursuit rollouts and already-trained compatible policies to increase state coverage during pretraining. After reconstruction training, the encoder weights are transferred into the RL feature extractor and frozen while the policy is trained. This produces an ablation between end-to-end image learning and fixed latent representations learned before RL.

4.5 UNet Encoder Variant

In addition to the CNN autoencoder, a UNet-style autoencoder is used for BEV representation learning. The encoder uses repeated double-convolution blocks with max pooling, followed by global average pooling to produce a compact 128-dimensional image descriptor. During pretraining, the full UNet decoder retains skip connections to improve image reconstruction quality; for RL, only the encoder path is kept and concatenated with the 64-dimensional vector embedding.

This second pretrained encoder provides a stronger structural prior than the plain CNN autoencoder. It is intended to test whether preserving multi-scale spatial detail during representation learning improves downstream control performance, particularly in scenes where obstacle shape and lane boundaries must both be interpreted from the BEV image.

4.6 Lidar Observation

The obstacle environments expose a simulated planar lidar observation by appending normalised obstacle distances to the state vector. In forward driving the lidar pose is attached to the tractor, while in reverse driving it is attached to the trailer and rotated by π so that the sensing direction follows the leading unit. This design keeps the sensing model consistent with the physical task: the front of the articulated system changes when the direction of travel reverses.

4.7 Observation Space Summary

The full observation interface used across experiments is summarised in Table 4.1.

Table 4.1: Observation space components

Component	Description	Role
`vector`	8-dimensional state: $[\delta, \gamma, e_y, e_\psi, e_{y,t}, e_{\psi,t}, \kappa_1, \kappa_2]$	Provides precise articulation and path-tracking state information
`lidar`	N normalised obstacle ranges, typically $N \in \{8, 16, 24\}$	Encodes local free space around the leading unit without image processing
`image`	84×84 grayscale BEV image rendered from Pygame simulation	Encodes lane layout, local path shape, obstacle positions, and vehicle geometry

Collision detection is performed using Pygame sprite mask overlap, enabling pixel-precise detection of contact between the vehicle body and obstacles or lane boundaries. Episode termination is triggered on collision, jackknife ($|\gamma| > 90°$), or path completion.

Chapter 5

Reinforcement Learning Framework

5.1 Twin Delayed Deep Deterministic Policy Gradient (TD3)

Twin Delayed Deep Deterministic Policy Gradient (TD3) is an off-policy actor-critic algorithm designed for continuous action spaces. TD3 extends deterministic policy-gradient methods with three key improvements: twin critic networks (to reduce overestimation bias), delayed policy updates (to improve stability), and target policy smoothing (to reduce variance in critic updates).

The TD3 actor network $\mu(s \mid \theta^{\mu})$ maps states to deterministic actions, while the critic networks $Q_1, Q_2(s, a \mid \theta^{Q_1}, \theta^{Q_2})$ evaluate action quality. The target value is computed using the minimum of the two critic estimates:

$$y = r_t + \gamma_d \min_{i=1,2} Q_i(s_{t+1},\, \mu(s_{t+1}) + \epsilon) \tag{5.1}$$

where $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$ is clipped Gaussian noise added to the target action for smoothing. Parameters are updated by minimising the temporal difference error in the critics and maximising the expected return through the actor using the deterministic policy gradient theorem.

TD3 is implemented using the Stable Baselines3 library, which provides a reliable implementation integrated with the Gymnasium environment API.

5.1.1 Reward Function

The proposed reward function is:

$$R = w_1 e^{-|e_y|} + w_2 e^{-|e_\psi|} - w_3|\gamma| - w_4 C \tag{5.2}$$

where:

- e_y = trailer cross-track error
- e_ψ = heading error
- γ = articulation angle
- C = collision penalty

A multiplicative alternative is also considered:

$$R = e^{-|e_y|} \cdot e^{-|e_\psi|} \tag{5.3}$$

5.1.2 Multiplicative vs. Additive Reward Structures

The multiplicative reward structure forces the agent to minimise cross-track and heading errors jointly, preventing the policy from exploiting partial credit in an additive form. In this thesis, that logic is extended to include hitch articulation stability: the multiplicative penalty on $|\gamma|$ discourages the agent from trading path accuracy for a large articulation angle.

5.2 Hyperparameter Optimisation

The TD3 agent requires careful hyperparameter tuning. The actor-critic learning rates, replay buffer size, target network update rate, and noise scale are selected over simulation rollouts. The training pipeline supports matched agents across state-only, lidar, and BEV observation variants. BEV policies use custom multi-input feature extractors, while the state-only and lidar policies use standard multilayer perceptron policies.

To reduce wall-clock cost for image-based training, the training script supports vectorised environment execution with subprocess workers. When multiple environments are used, the data collection schedule switches from episode-based to step-based updates so that parallel workers remain saturated instead of waiting for all episodes to terminate simultaneously. This is particularly important for the CNN-based BEV policy, whose throughput is otherwise dominated by environment interaction rather than gradient computation.

5.2.1 Representation Pretraining

Two of the BEV variants use an explicit pretraining stage before TD3 optimisation. The autoencoder-based model freezes a pretrained NatureCNN encoder, while the UNet-based model freezes a pretrained UNet encoder after transfer from the reconstruction network. Both pretraining procedures operate on BEV images collected from policy rollouts and

simple geometric controllers. This separates representation learning from control learning and allows direct comparison against the end-to-end CNN baseline.

5.2.2 Classical Controller Tuning

A reproducible tuning pipeline is used for the classical baselines in later comparison chapters. Forward and reverse pure-pursuit, PID, and MPC controllers are optimised using differential evolution over repeated simulation episodes. The scalar objective penalises trailer cross-track error, jackknife events, collisions, and non-completion. After optimisation, each controller is validated on held-out seeds and a two-dimensional sensitivity sweep is generated over the most influential parameters. This ensures that comparisons against TD3 are made against tuned baselines rather than hand-picked nominal gains.

5.3 Failure Replay Curriculum

A persistent challenge in training RL agents on the reverse lane-following task is that the episode failure rate is high early in training and the lane geometry is re-randomised at every reset. In the standard random-reset regime, each failure is discarded and replaced by a fresh, randomly drawn layout. As a result, the agent rarely encounters the same hard configuration twice, reducing the pressure to master any particular geometry and potentially slowing convergence on the inherently open-loop-unstable reverse task.

To address this, a *failure replay curriculum* is implemented as a transparent environment wrapper. At each episode boundary the wrapper inspects the episode outcome: if the episode terminated due to a failure—jackknife, collision, or out-of-bounds departure before reaching the goal—the next reset reuses the same random seed that generated the current path and spawn position, exactly replaying the same scenario. If the episode was completed successfully or timed out, a fresh seed is drawn and a new random layout is used. A configurable consecutive-retry ceiling (default five retries) prevents the agent from being trapped indefinitely on a pathologically difficult layout.

This strategy is a form of automatic curriculum learning: rather than constructing a manually graded task sequence, the curriculum emerges from the agent's own failure signal. It is implemented orthogonally to the reward formulation and observation representation ablations. The wrapper modifies only the episode-level sampling policy—not the per-step reward signal or the information available to the agent—and is therefore controlled by a separate training flag. Models trained with and without failure replay are saved to distinct directories to ensure clean comparison.

The curriculum is most meaningful for scenarios where the per-episode failure rate is high and layout difficulty is non-uniform, namely reverse lane-following and reverse obstacle avoidance. For forward lane-following the failure rate is low after minimal training

and the benefit is expected to be negligible.

Chapter 6

Simulation Environment

6.1 Custom Pygame/Gymnasium Environment

The training and evaluation environment is a custom 2D simulation built on the Farama Foundation Gymnasium library. Gymnasium provides a standardised API for implementing environments that integrate directly with reinforcement learning algorithm libraries, including Stable Baselines3.

The simulation is implemented using Pygame for rendering and physics integration. At each timestep, the discrete-time tractor-trailer state-space equations are re-evaluated using the previous state and control inputs, updating the vehicle position, heading, hitch angle, and associated error signals. Pygame renders the scene as a 2D top-down view, from which the BEV image observation is captured.

The simulation environment includes:

1. **Vehicle Model**: A combined dynamic tractor (bicycle model with Pacejka tire dynamics) and kinematic trailer with hitch constraints, using Tesla Model S parameters.

2. **Lane Generation**: Randomly generated lane paths using cubic spline interpolation, producing varying curvature paths for training and evaluation.

3. **Obstacle Placement and Local Replanning**: Randomly placed static obstacles within the lane and a local path planner that perturbs the centreline around those obstacles for avoidance experiments.

4. **Collision Detection**: Pygame sprite mask-based collision detection, providing pixel-precise contact sensing between the vehicle body and obstacles or lane boundaries.

5. **BEV Camera**: A configurable bird's-eye view camera anchored to the tractor's rear axle, rendering an 84×84 grayscale image of the vehicle and its surroundings.

6.2 Environment Tasks

The environment implementation is structured around reusable task and observation components. A base lane-following task is combined with different observation heads (state only, lidar, or BEV), while obstacle-aware variants reuse the same task logic through a mixin that adds obstacle generation, local path planning, and obstacle-aware reward evaluation. This avoids maintaining separate forward, reverse, lidar, and BEV implementations with duplicated task logic.

6.2.1 Line Following

The lane-following environment extends the base tractor-trailer environment to present a lane-tracking task. A reference lane is generated at the start of each episode using cubic spline interpolation through randomly placed waypoints. The reward function encourages the agent to keep both the tractor and trailer within the lane boundaries while maintaining progress along the path.

For forward driving, the observation at each timestep may include:

- `vector`: steering angle δ, hitch articulation angle γ, tractor and trailer cross-track and heading errors, and local curvature cues;
- `lidar`: simulated obstacle distances appended to the vector state when lidar sensing is enabled;
- `image`: an 84×84 grayscale BEV image rendered from the tractor rear axle anchor point.

Reverse line-following environments use the same path generation and rendering pipeline, but reverse the longitudinal command and compute reward and tracking terms with the trailer as the leading unit. Reverse BEV and reverse lidar observation classes are implemented explicitly so that perception experiments can be repeated in backward manoeuvres without changing the task definition.

6.2.2 Obstacle Avoidance

The obstacle avoidance environment adds static obstacles within the lane. The agent must navigate the tractor-trailer through the lane while avoiding obstacles, which requires both path-following behaviour and collision avoidance. Instead of following the global lane centreline directly, the environment constructs a local path around the placed obstacles and evaluates tracking error relative to this locally planned path. Episode termination occurs on collision, jackknife ($|\gamma| > 90°$), or successful path completion.

Forward and reverse obstacle environments are both available. In addition to the lidar-based formulation, the environment includes BEV obstacle-avoidance variants for both travel directions, enabling direct comparison between range-based and image-based obstacle perception within the same local-planning task.

6.3 Vehicle Parameters

The simulation uses Tesla Model S specifications for the tractor parameters:

Table 6.1: Vehicle parameters used in simulation

Parameter	Value	Parameter	Value
m	$1500\,\mathrm{kg}$	l_f	$1.2\,\mathrm{m}$
I_z	$3000\,\mathrm{kg\,m^2}$	l_r	$1.6\,\mathrm{m}$
C_f	$80000\,\mathrm{N/rad}$	C_d	0.208
C_r	$80000\,\mathrm{N/rad}$	A	$2.4\,\mathrm{m^2}$
Δt	$0.1\,\mathrm{s}$	ρ	$1.225\,\mathrm{kg/m^3}$

6.4 Action Space

The agent controls the vehicle through a continuous two-dimensional action space:

$$\mathbf{a} = [\dot{\delta},\, v] \in [-15^\circ/\mathrm{s},\, 15^\circ/\mathrm{s}] \times [0,\, 2\,\mathrm{m/s}] \tag{6.1}$$

where $\dot{\delta}$ is the steering rate (rate of change of steering angle) and v is the commanded speed. This action parameterisation reflects the fact that the steering angle cannot be set instantaneously on a physical vehicle; controlling the rate of change is more physically realistic.

For reverse tasks, the environment constrains the longitudinal command to negative speed while preserving the same steering-rate interface. This keeps the control dimensionality fixed across forward and reverse tasks and allows the same TD3 implementation to be reused.

For evaluation, this full action space is not used identically in every study. In the no-obstacle line-following experiments, the longitudinal speed is treated as a fixed operating condition so that all methods are compared on the same path-tracking problem. In the obstacle-avoidance experiments, speed modulation remains part of the task because the agent must trade progress against collision risk and path geometry.

6.5 Testing Protocols

The experimental workflow is now scriptable end-to-end. Scenario generation, Phase 1 observation-modality training, trained-policy evaluation, controller tuning, and multi-controller benchmarking are each exposed through standalone scripts. The benchmarking pipeline runs all controllers on the same pre-generated scenario set and logs per-step metrics including tractor and trailer cross-track error, hitch angle, completion, collision, jackknife events, obstacle clearance, and inference latency.

The experiments are treated as two related but distinct problem classes:

1. **Path-tracking control**: forward and reverse lane following without obstacles. In this setting all methods are solving the same task at a common fixed speed, so TD3 can be compared directly against pure-pursuit, PID, and MPC as lateral/path-tracking controllers.

2. **Integrated navigation**: obstacle-avoidance tasks in which the policy must implicitly combine local planning and control. In this setting a direct comparison against controller-only baselines is not fair unless those baselines are also given a local planner.

This distinction is important for experimental validity. A controller that only tracks a supplied path should not be evaluated against an RL policy that must both choose a collision-free local path and execute it, unless the classical baseline is paired with an equivalent planner.

6.5.1 Sim-to-Real and ROS2 Transition Strategy

Although the experiments in this thesis are simulation-only, the software architecture is intended to support later deployment in a ROS2-based robotics stack. The key design decision is that the policy and controller interfaces are expressed in terms of generic observations and low-level control commands rather than simulator-specific rendering calls. Each control method consumes either a vector observation, a lidar-style range vector, or a BEV image paired with state variables, and produces steering-rate and speed commands. This abstraction is directly compatible with a ROS2 node graph in which sensor drivers publish observations, a policy node performs inference, and a downstream interface node converts the commands into actuator messages.

For no-obstacle path-tracking experiments, the fixed-speed formulation used in the thesis also improves deployment clarity: a real vehicle implementation can treat longitudinal speed regulation as a separate closed-loop subsystem while the learned or analytical controller handles lateral and articulation control. For obstacle-navigation experiments, the policy retains speed authority because longitudinal modulation is part of the task

definition. This separation reduces ambiguity when mapping the thesis experiments onto a real robotic architecture.

The expected ROS2 transition therefore consists of replacing only the environment-facing components:

- the simulated state vector with subscribed vehicle-state estimates;
- the simulated lidar or BEV renderer with ROS2 sensor topics or perception outputs;
- the Pygame environment step call with a command publisher targeting steering and speed interfaces;
- the episode reset logic with scenario initialisation and safety supervision on the robot.

The policy network, observation formatting, benchmark metrics, and controller-selection logic can remain largely unchanged. This is important because it turns ROS2 deployment from a full reimplementation problem into an interface-substitution problem.

However, portability should not be overstated. Before deployment, the simulator must be augmented with domain randomisation and system identification so that the learned policy is exposed to uncertainty in tire parameters, friction, mass distribution, actuator delay, sensor noise, and hitch compliance. In the absence of these steps, strong simulation performance alone is not evidence of hardware readiness.

6.5.2 Lane Following Performance

Table 6.2: Lane following experiment definitions

Experiment	Objective	Success Metric
Path Fidelity (Tractor)	Maintain low CTE for tractor on varying curvature paths	Mean Absolute CTE
Path Fidelity (Trailer)	Maintain low CTE for trailer, ensuring full vehicle stays in lane	Mean Absolute CTE
Hitch Stability	Keep articulation angle within safe bounds	Max $\|\gamma\|$ per episode

6.5.3 Obstacle Avoidance

The obstacle avoidance task tests the agent's ability to navigate through a lane containing static obstacles. Success is measured by collision rate and the agent's ability to complete

the lane while maintaining hitch stability.

Obstacle experiments are framed as a secondary study rather than merged into the primary controller benchmark. Two defensible comparison strategies are available:

- **Planner-controlled comparison**: provide the classical baselines with the same obstacle-aware local path and evaluate only the downstream tracking problem; or
- **End-to-end navigation comparison**: compare the RL obstacle policy against complete planner+controller stacks, not against controllers in isolation.

In the present study, obstacle avoidance is treated as an RL feasibility extension and the main quantitative comparison is reserved for the no-obstacle forward and reverse tracking tasks. Under this structure, no-obstacle experiments use fixed speed for both learned and classical methods, whereas obstacle experiments allow the RL agent to command speed as part of the end-to-end navigation problem.

Chapter 7

Results and Discussion

7.1 Path Following Performance

The results pipeline supports controlled comparison across observation modalities and controller families. Forward lane-following serves as the first benchmark task, with matched TD3 agents trained on state-only, lidar, end-to-end CNN BEV, pretrained autoencoder BEV, and pretrained UNet BEV observations. The evaluation script aggregates tractor and trailer cross-track error, hitch-angle statistics, completion rate, and inference latency across repeated episodes.

This structure is important for the thesis because it separates two questions that were previously entangled: whether the articulated control task benefits from richer sensing, and whether any benefit comes from the sensing modality itself or from the representation-learning pipeline used to process it.

The experiments are organised in the following order:

1. select the best observation space on forward and reverse lane-following without obstacles;

2. hold that observation fixed and compare reward formulations on the same no-obstacle tasks;

3. benchmark the best TD3 configuration against tuned classical controllers on matched no-obstacle scenarios;

4. evaluate the failure replay curriculum (Section **??**) on the reverse task, holding observation and reward fixed, to isolate the effect of the reset strategy on convergence and final performance.

This sequence keeps each ablation focused on one design variable at a time and ensures that the final controller benchmark is performed on the cleanest apples-to-apples task definition. In particular, the no-obstacle benchmark is run at fixed speed for every

method, so that differences in performance are attributed to path tracking and articulation stabilisation rather than to longitudinal speed selection.

Table 7.1 converts that sequence into an executable run matrix. The training launcher supports all observation and reward combinations listed there through the `scenario`, `obs`, and `reward` arguments, while the benchmark scripts are used only after model selection.

Table 7.1: Planned run matrix for the experimental campaign

Study	Scenario	Observation sweep	Reward sweep	Runs	Purpose
Phase 1A	forward	state, lidar, BEV	dense	3	Select best forward observation space
Phase 1B	reverse	state, lidar, BEV	dense	3	Select best reverse observation space
Phase 2A	forward	best from Phase 1A	dense, tractor_focus, multiplicative	3	Select best forward reward
Phase 2B	reverse	best from Phase 1B	dense, no_hitch, multiplicative	3	Select best reverse reward
Phase 3	forward, reverse	best RL configuration per direction	benchmark only	2	Compare TD3 against tuned pure pursuit, PID, and MPC
Phase 4	forward_obs, reverse_obs	lidar or BEV	best reward from matching direction	4	Secondary obstacle-navigation study

7.2 Observation Space Ablation

The observation space ablation compares seven configurations on the no-obstacle forward and reverse tasks: state-only, lidar (best beam count from Section **??**), end-to-end CNN BEV from random initialisation, frozen autoencoder BEV, fine-tuned autoencoder BEV, frozen UNet BEV, and fine-tuned UNet BEV. All agents share the same TD3 backbone, reward formulation, and hyperparameters, so performance differences are attributed to

the observation and representation choices alone.

Figure 7.1: Learning curves (mean episode return ± 1 s.d.) for forward lane-following: observation space ablation. Shaded bands show one standard deviation across evaluation episodes at each checkpoint.

Figure 7.2: Learning curves for reverse lane-following: observation space ablation.

Table 7.2: Forward lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).

Observation	CTE					
Tractor						
(m)	CTE					
Trailer						
(m)	Max $\lvert\gamma\rvert$					
(°)	Completion					
(%)	Jackknife					
(%)	Inference					
(ms)						
State (MLP)	—	—	—	—	—	—
Lidar-16 (MLP)	—	—	—	—	—	—
BEV CNN (scratch)	—	—	—	—	—	—
BEV AE (frozen)	—	—	—	—	—	—
BEV AE (unfrozen)	—	—	—	—	—	—
BEV UNet (frozen)	—	—	—	—	—	—
BEV UNet (unfrozen)	—	—	—	—	—	—

Table 7.3: Reverse lane-following: observation space ablation (dense reward, no obstacles, 30 eval episodes).

Observation	CTE					
Tractor						
(m)	CTE					
Trailer						
(m)	Max $\lvert\gamma\rvert$					
(°)	Completion					
(%)	Jackknife					
(%)	Inference					
(ms)						
State (MLP)	—	—	—	—	—	—
Lidar-16 (MLP)	—	—	—	—	—	—
BEV CNN (scratch)	—	—	—	—	—	—
BEV AE (frozen)	—	—	—	—	—	—
BEV AE (unfrozen)	—	—	—	—	—	—
BEV UNet (frozen)	—	—	—	—	—	—
BEV UNet (unfrozen)	—	—	—	—	—	—

7.3 Lidar Beam-Count Ablation

Lidar is treated separately from image-based methods because it is a qualitatively different sensing modality: a sparse one-dimensional range scan rather than a two-dimensional bird's-eye-view image. The beam-count ablation isolates the effect of lidar resolution on path-tracking performance and inference cost. Four beam counts are compared (4, 8, 16, 32) on the forward no-obstacle task with the dense reward formulation.

Table 7.4: Lidar beam-count ablation on forward lane-following (dense reward, no obstacles, 30 eval episodes). The 16-beam result is the same run as in Table **??**.

Beams	CTE Trailer mean (m)	CTE Trailer RMS (m)	Max $\lvert\gamma\rvert$ (°)	Completion (%)	Jackknife (%)	Inference (ms)
4	—	—	—	—	—	—
8	—	—	—	—	—	—
16	—	—	—	—	—	—
32	—	—	—	—	—	—

7.4 Reward Formulation Ablation

The reward ablation holds the observation space fixed at the best configuration from Section **??** and compares reward formulations. Four variants are evaluated on forward lane-following and four on reverse lane-following.

Figure 7.3: Learning curves for forward lane-following: reward formulation ablation.

Figure 7.4: Learning curves for reverse lane-following: reward formulation ablation.

Table 7.5: Forward lane-following: reward formulation ablation (state obs, no obstacles, 30 eval episodes).

Reward	Tractor CTE (m)	Trailer CTE (m)	Max $\lvert\gamma\rvert$ (°)	Completion (%)	Jackknife (%)	Total reward
Dense	—	—	—	—	—	—
Tractor focus	—	—	—	—	—	—
Multiplicative	—	—	—	—	—	—
Guided	—	—	—	—	—	—

Table 7.6: Reverse lane-following: reward formulation ablation (state obs, no obstacles, 30 eval episodes).

Reward	Tractor CTE (m)	Trailer CTE (m)	Max $\lvert\gamma\rvert$ (°)	Completion (%)	Jackknife (%)	Total reward
Dense	—	—	—	—	—	—
No hitch	—	—	—	—	—	—
Multiplicative	—	—	—	—	—	—
Guided	—	—	—	—	—	—

7.5 Statistical Rigor and Generalisation

The final reported results are not limited to single aggregate scores. For each ablation and benchmark comparison, the thesis reports the number of training seeds, the number

of held-out evaluation scenarios, and a measure of uncertainty such as standard deviation or confidence intervals. Where two methods are close in mean performance, effect sizes or simple statistical tests distinguish meaningful differences from noise induced by stochastic training and random scenario generation.

Generalisation is evaluated explicitly rather than assumed. In the present simulator, this means constructing held-out benchmark sets that differ from the training distribution in controlled ways, for example through tighter curvature segments, different spawn offsets, unseen reverse initialisation errors, denser obstacle placements, or degraded observations. A controller that performs well only on nominal paths does not provide strong evidence of robustness for articulated autonomy. The results therefore separate *in-distribution* benchmark performance from *out-of-distribution* robustness tests.

7.6 Training Strategy: Failure Replay Curriculum

The failure replay curriculum (Section **??**) is evaluated as a controlled ablation on the reverse lane-following task, where the episode failure rate is highest and the effect of replay is therefore expected to be most pronounced. Two agents are trained with identical hyperparameters, observation spaces, and reward formulations, differing only in whether the reset strategy replays failed scenarios or draws a fresh random layout.

The primary comparison metric is the learning curve: mean episode return and completion rate plotted against training timesteps. A curriculum that accelerates convergence produces a steeper rise in completion rate during the early phase of training, when the policy is least stable and failure rates are highest. An effective replay curriculum reaches a given completion threshold in fewer timesteps than the random-reset baseline.

Final performance (mean absolute trailer CTE, jackknife rate, and max $|\gamma|$ over held-out evaluation seeds) is also compared, as the curriculum may improve sample efficiency without necessarily raising the asymptotic performance ceiling. Results are evaluated on fixed held-out seeds that were not seen during training, so that any advantage from repeated exposure to particular layouts during training does not inflate the reported evaluation metrics.

Figure 7.5: Learning curves for reverse lane-following: failure replay curriculum vs random reset baseline. Both agents use identical hyperparameters, observation space, and reward formulation.

Table 7.7: Failure replay curriculum ablation on reverse lane-following (state obs, dense reward, 30 held-out eval episodes). Learning curves shown in Figure **??**.

Reset strategy	Steps to 50% completion ($\times 10^3$)	CTE (m)	Max $\lvert\gamma\rvert$ (°)	Completion (%)	Jackknife (%)
Random reset	—	—	—	—	—
Failure replay	—	—	—	—	—

7.7 Reverse Maneuvering Success

Reverse evaluation uses dedicated reverse line-following and reverse obstacle-avoidance environments, along with reverse pure-pursuit, reverse PID, and reverse MPC baselines. All reverse controllers are formulated around hitch-angle stabilisation and trailer-referenced tracking errors, reflecting the fact that the trailer leads the manoeuvre during backing. This provides a consistent basis for comparing learned and analytical controllers on the inherently unstable reverse articulated-driving problem.

7.8 Perception Robustness

The perception study uses a concrete ablation structure: state variables only, state plus lidar, end-to-end BEV learning, and frozen pretrained BEV encoders. Because all variants share the same TD3 backbone and task definition, differences in tracking performance can be attributed more directly to the observation and representation choices than to changes in the control architecture.

In practical terms, the most meaningful observation-space study is the one run on the no-obstacle environments. Obstacle environments confound the comparison because different observation spaces change not only path-tracking quality but also how much implicit local-planning information is available to the policy. They also change how much benefit a method can obtain from modulating speed, which is undesirable in the core controller comparison.

7.9 Obstacle Navigation

Obstacle-avoidance results are reported separately from the no-obstacle benchmark because the RL agent learns a joint local-planning and control policy (including speed modulation), while the classical baselines are pure path-tracking controllers. The comparison is therefore not apples-to-apples and is interpreted as an end-to-end navigation study rather than a controller-only comparison.

Table 7.8: Forward obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).

Observation	CTE						
Trailer							
(m)	Max $\lvert\gamma\rvert$						
(°)	Path						
completion							
(%)	Collision						
(%)	Jackknife						
(%)	Min clearance						
(m)	Inference						
(ms)							
State (MLP)	—	—	—	—	—	—	—
Lidar-16 (MLP)	—	—	—	—	—	—	—
BEV CNN (scratch)	—	—	—	—	—	—	—

Table 7.9: Reverse obstacle-avoidance: TD3 performance by observation space (dense reward, 5–10 obstacles per episode, 30 eval episodes).

Observation	CTE						
Trailer							
(m)	Max $\|\gamma\|$						
(°)	Path						
completion							
(%)	Collision						
(%)	Jackknife						
(%)	Min clearance						
(m)	Inference						
(ms)							
State (MLP)	—	—	—	—	—	—	—
Lidar-16 (MLP)	—	—	—	—	—	—	—
BEV CNN (scratch)	—	—	—	—	—	—	—

7.10 Benchmark Comparison

Table 7.10: Forward lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).

Controller	CTE					
Trailer						
mean (m)	CTE					
Trailer						
RMS (m)	Max $\|\gamma\|$					
(°)	Completion					
(%)	Jackknife					
(%)	Latency					
(ms)						
TD3 (best config)	—	—	—	—	—	—
Forward pure pursuit	—	—	—	—	—	<1
PID (tuned)	—	—	—	—	—	<1
MPC (tuned)	—	—	—	—	—	—

Table 7.11: Reverse lane-following: controller benchmark (no obstacles, 50 eval episodes, held-out seeds).

Controller	CTE Trailer mean (m)	CTE Trailer RMS (m)	Max $\lvert\gamma\rvert$ (°)	Completion (%)	Jackknife (%)	Latency (ms)
TD3 (best config)	—	—	—	—	—	—
Reverse pure pursuit	—	—	—	—	—	<1
PID reverse (tuned)	—	—	—	—	—	<1
MPC reverse (tuned)	—	—	—	—	—	—

Benchmarking is performed on pre-generated scenario sets so that TD3 and the classical baselines face identical lane geometry and obstacle placement. Classical controllers are tuned before comparison using differential evolution and then validated on held-out seeds. Each benchmark episode logs both control quality and computational cost, allowing comparison not only of tracking error but also of completion rate, jackknife frequency, collision rate, obstacle clearance, and controller latency.

The results must be understood in terms of multi-objective performance: in rigid-body systems, success is synonymous with minimising cross-track error. In articulated systems, hitch stability takes precedence over exact path following. An agent that maintains $|\gamma| < 5°$ throughout a reverse task is more successful than one that achieves lower CTE but triggers jackknife events. For that reason, the benchmark framework reports trailer tracking, articulation stability, and completion together rather than reducing performance to a single scalar.

For obstacle scenarios, the benchmark question changes. An RL obstacle policy is effectively learning a joint local-planning and control policy, including speed modulation, whereas pure-pursuit, PID, and MPC are primarily path-tracking controllers. Obstacle-task results are therefore not presented as directly comparable to controller-only baselines unless a matched planning layer and longitudinal policy are added to those baselines. Obstacle results are reported either as a separate end-to-end navigation study or as a comparison against planner+controller classical stacks.

7.11 Failure Analysis and Deployment Relevance

Average metrics alone are insufficient for articulated-vehicle control, because the operational significance of a failure depends strongly on how that failure occurs. Representative failure cases for each controller family are included, such as gradual corner-cutting, growing lateral oscillations, abrupt jackknife onset, and obstacle-clearance failures. Trajectory overlays and hitch-angle traces are particularly valuable because they show whether an error mode is primarily geometric, dynamic, or perception-driven.

This failure-oriented discussion also connects the simulation study to deployment relevance. A controller that attains a low mean cross-track error but exhibits rare catastrophic jackknife episodes is less deployable than a controller with slightly higher average error but consistent stability margins. For the same reason, controller latency and architectural modularity are discussed alongside tracking performance. These considerations help position the thesis as a study in autonomous articulated-system design rather than only an optimisation exercise inside a simulator.

Table 7.12: Recommended experiment matrix for the thesis

Experiment	Task	Observation	Success Metric
Observation ablation (forward)	Path following	state, lidar, BEV	Fixed-speed trailer mean abs. CTE, max $\|\gamma\|$, completion
Observation ablation (reverse)	Reverse tracking	state, lidar, BEV	Fixed-speed trailer mean abs. CTE, max $\|\gamma\|$, jackknife rate
Reward ablation (forward)	Path following	best observation only	Fixed-speed mean return, trailer CTE, stability metrics
Reward ablation (reverse)	Reverse tracking	best observation only	Fixed-speed completion, jackknife rate, trailer CTE
Controller benchmark (forward)	Path following	best RL obs. vs. classical	Fixed-speed TD3 vs. pure pursuit vs. PID vs. MPC
Controller benchmark (reverse)	Reverse tracking	best RL obs. vs. classical	Fixed-speed TD3 vs. reverse pure pursuit vs. reverse PID vs. reverse MPC
Obstacle RL extension	End-to-end navigation	lidar or BEV	Variable-speed collision rate, completion, min clearance
Obstacle classical comparison	Planner + controller	matched local path + controller	Only if classical baselines receive equivalent planning information

Chapter 8

Conclusion

8.1 Summary of Findings

This thesis presented a reinforcement learning framework for articulated autonomous vehicle control. The key contributions demonstrated are:

- a combined dynamic tractor and kinematic trailer model implemented in a custom Gymnasium simulation environment;
- a vision-based observation space incorporating BEV images rendered from the Pygame simulation, enabling spatial perception of lane boundaries and obstacles;
- a TD3 reinforcement learning agent trained on lane-following and obstacle avoidance tasks with a multiplicative reward function penalising cross-track error, heading error, and hitch articulation angle;
- comparison of TD3 performance against tuned pure-pursuit, PID, and MPC classical control baselines on matched articulated path-tracking tasks.

The integration of visual perception via BEV imagery broadens the control problem beyond compact vector-state tracking. By processing rendered BEV images through a CNN feature extractor, the agent gains spatial context about lane geometry and obstacle positions that vector state observations alone cannot provide.

From a research-impact perspective, the thesis is strongest when interpreted not only as an RL training exercise but as a reproducible benchmark and systems study for articulated autonomy. The combination of matched observation ablations, tuned classical baselines, fixed-scenario benchmarking, and explicit articulation-stability metrics provides a framework for asking which sensing and control choices are actually useful for tractor-trailer path tracking. This framing is more defensible and more reusable than reporting isolated policy demonstrations.

8.2 Limitations

The study is subject to the following constraints:

- Experiments are conducted entirely within the custom 2D Pygame/Gymnasium simulation environment; performance in higher-fidelity simulators or on physical hardware has not yet been characterised.
- The 2D simulation approximates the vehicle dynamics but does not capture all effects present in a real tractor-trailer system, such as load transfer, tire saturation at high slip angles, or hitch joint compliance.
- The current framework assumes a fixed payload; hitch dynamics change significantly with varying trailer mass.
- Obstacle avoidance is limited to static obstacles; dynamic obstacle handling is not addressed.
- Generalisation beyond the training distribution has not yet been fully quantified; robustness to sensor degradation, actuator delay, and stronger path-distribution shift remains to be measured explicitly.
- The current thesis demonstrates simulator portability of the software abstractions, but not yet end-to-end ROS2 deployment or sim-to-real transfer on hardware.

8.3 Future Work

Several directions remain open for future investigation:

- **Higher-fidelity simulation**: Validating the trained policies in a physics-based 3D simulator (e.g., Gazebo) with a detailed URDF model of the tractor-trailer system, including realistic tire and hitch dynamics.
- **ROS2 deployment pipeline**: Packaging the policy and controller interfaces as ROS2 nodes, defining the sensor-topic and actuator-command contracts explicitly, and validating that the simulator-trained stack can be executed without algorithmic redesign in a robotics middleware setting.
- **Hardware deployment**: Transferring trained policies to a physical robotic platform, requiring sim-to-real transfer techniques such as domain randomisation, calibrated system identification, latency compensation, and supervised safety fallbacks.

- **Planner-controller baselines for obstacle tasks**: Comparing obstacle-avoidance RL policies against complete classical navigation stacks, so that end-to-end navigation is evaluated on a fair task definition rather than against controller-only baselines.
- **Robustness benchmarks**: Adding out-of-distribution test sets with stronger curvature, perturbed initial poses, observation noise, and parameter mismatch, so that the thesis can quantify generalisation rather than only nominal performance.
- **Failure-mode analysis**: Building a structured taxonomy of controller failures, including oscillation growth, corner-cutting, hitch divergence, and obstacle-clearance collapse, to support safety-oriented comparison rather than relying on averages alone.
- **Sequence-model policies for tractor-trailer control**: Applying transformer-based offline control methods to the articulated vehicle setting, with training data generated from the TD3 expert policy and augmented with high-error recovery trajectories.
- **Multi-trailer systems**: Extending the kinematic model and RL policy to double or B-train configurations.
- **Online adaptation**: Updating the policy in real time to compensate for varying payloads and road conditions.
- **Control Barrier Function overlays**: Implementing CBFs as a hard safety layer that overrides the RL agent when a collision or jackknife is imminent.
- **Dynamic obstacles**: Extending the obstacle avoidance environment to include moving obstacles and integrating predictive collision avoidance.

Appendix A

Simulation Software Architecture

Appendix B

Full Hyperparameter Tables

Table B.1: TD3 Hyperparameters

Hyperparameter	Value
Learning rate	
Batch size	
Replay buffer size	
Policy noise	
Target update rate	
Timesteps	200,000

Appendix C

Additional Plots

Appendix D

Reward Function Derivation

Bibliography